Plan: GömbSoma optimization passes 5–8

Targeted speedup of the remaining 28 ms per-image cost

2026-05-15

1 Background

Passes 1–4 took the GömbSoma `Config E` per-image forward from 8 283 ms to 28.0 ms (296×). The remaining cost is spread across many small components rather than a single dominant hot spot:

Component	ms	% of total
StimulusGraphBuilder (×2 in Config E)	9–10	33
"Unaccounted"	10–11	37
Quadtree	2.2	8
SDRF	3.0	11
Bochner × 3 branches	1.4	5
Patch encoder	0.2	1

2 Scope

Four targeted optimisation passes, each addressing one slice of the remaining cost. None touches CORE.YAML-protected files. Each is independently revertable. The 150-test Ricci-Stim suite must pass after every pass.

2.1 P5 — Profile the "unaccounted" 10 ms

Hypothesis: sparse-tensor coalesce calls on M_v constructions plus Python dispatch overhead in the per-image batch loop.

Fix: use `torch.profiler` (kineto) to capture per-op timing on a single forward pass at the Config E shape. Identify which specific ops eat the 10 ms.

Expected: 10 ms → 3 ms (save 7 ms) once the hot ops are identified and replaced with cheaper equivalents.

Risk: the unaccounted may be many small ops with no single fix. If so, fall back to a sub-ms profile and accept the 10 ms as fundamental Python orchestration overhead, blocked on P8.

2.2 P6 — Cache topology between SDRF passes 1 and 2

Hypothesis: `StimulusGraphBuilder` is called twice in Config E. Both calls use the same quadtree, so:

- `_same_scale_edges` produces byte-identical output
- `_cross_scale_edges` produces byte-identical output
- adjacency dict is identical

Only the edge *signs* and the SDRF-added shortcut edges differ between calls.

Fix: add a `cached_topology` parameter to `StimulusGraphBuilder.forward`; pass 1 caches; pass 2 reuses the cached topology and only computes signs + the augmented walks / polygons / triangles incident to the new edges.

Expected: 9.5 ms → 3 ms on the second pass. Total Config E save: 6 ms.

Risk: caching wrong objects (e.g., tensors with stale gradients) would corrupt the forward. The cache key must be `(id(tree), tree.positions.data_ptr(), tree.sizes.data_ptr())` or equivalent.

2.3 P7 — Vectorise polygon enumeration

Hypothesis: the polygon plaquette enum at ∼200 polygons is still a Python loop over `(r, c, s)` tuples with a dict lookup per side. The vectorised walk / triangle enum did not help because (n, n, n) tensor allocation overhead matches the Python cost at $n = 256$. Polygons are simpler: just check four neighbors per anchor. The vectorisation pays off here because the per-anchor work is structured (top-left → right → bottom → diag).

Fix: sort anchors by (scale, r, c) so adjacency is stride-1; build a $(n_{\text{anchors}}, 4)$ neighbour-index tensor; check the plaquette condition via tensor masks.

Expected: 3–4 ms saved inside StimulusGraphBuilder.

Risk: edge-index lookup for plaquette edges must match the caller's edge ordering; mismatches caused a bug in P3 (σ-product test caught it). Same lookup-via-shared-dict pattern applies.

2.4 P8 — GPU-level batching across images

Hypothesis: the per-image Python forward (`for b in range(batch_size): outs.append(_forward_singl`
is fundamentally Python-bound. To break out, we need a SegmentedSparse representation where all images' anchors live in one big sparse graph and per-image offsets index them.

Fix:

1. Concatenate all images' anchor positions / sizes into single tensors, with a per-image offset array.
2. Concatenate quadtree-derived edges with per-image offset shifts (so anchor indices in image B don't collide with image A).
3. Run one big `StimulusGraphBuilder` on the concatenated graph.
4. Run one big Bochner-coupled `HypergraphConv` on the concatenated graph.
5. Split outputs back per-image at the head.

Expected: 5–10× on per-image-amortised cost. At batch_size = 8 this means 28 ms / image → 3–5 ms / image amortised.

Risk: multi-session task. Touches every module's forward signature. Plan as its own phase if the smaller fixes don't get us into the desired range.

3 Test strategy

After each pass:

- Full Ricci-Stim 150-test suite passes.
- Profile reruns: P5-target ms → measured ms shown in before/after table.
- Smoke run (30 imgs × 1 epoch × Config E) wall time recorded; expected monotone decrease.

4 Performance budget

Per-pass GPU time ≤ 0 (these are CPU/memory optimisations); RSS unchanged (we are not allocating new big tensors). The 16 GB cgroups cap applies to any post-optimisation training runs.

5 Rollback path

Each pass is a single-module diff. Revert via git revert of the specific commit. P5 and P6 are independent; P7 depends on P6's edge-lookup contract; P8 is a separate architectural phase.

6 Risk anticipation

- *Mid-ablation source modifications.* A source modification during a running ablation would taint the experimental comparison because Python re-imports between configs. **Strict rule:** no source changes until the ablation completes. This plan is applied AFTER the 5-config × 1-seed ablation (`ricci-stim-ablation-2026-05-15.service`) returns its full table.
- *Diminishing returns.* 296× already delivered; the next 10× requires P8 (architectural). P5–P7 alone get us perhaps 2× more (14 ms / image / 70 FPS).
- *Smoke regressions.* The smoke test (30 imgs × 1 epoch × Config E) is the canonical sanity gate after each pass. If it slows down, revert immediately.
- *Empty-plan-dir hygiene.* If P5 reveals there is no single big-win op in the unaccounted 10 ms and we shelve the plan, the plan dir is deleted before commit.

7 Decision tree

- **Config E in the ablation $\geq$ 0.5 mAP50_proxy:** proceed with P5–P7 to get 28 → 14 ms (70 FPS). P8 deferred unless training wall-time becomes the gate.
- **Config E in** $[0.235, 0.5)$**:** architecture is partly validated; P5–P7 + a second-seed run becomes the priority over raw FPS gains.
- **Config E** < 0.235**:** stimulus-driven hypothesis falsified; optimization passes are deferred while we reconsider the architecture.

8 Bottom line

Architecturally complete, empirically validated (or not — pending ablation), and now ready for a focused four-pass per-image speedup sweep. The plan is paper-only until the in-flight ablation lands; no source modifications mid-run.